Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Bases de Datos 1

AutoAudit

Sistema de auditoría automatizado

Documentación técnica

Diagrama entidad-relación · Diccionario de datos · Manual de configuración · Procedimiento de instalación y despliegue de triggers · Interfaz, pruebas y rendimiento

Proyecto final del curso

Estudiantes: Alan Daniel Jolón Zacarías y Bryan José Azañon Alvarado
Carné: 9989 – 22 – 9548 y 1090 – 22 – 21826
Catedrático: Edgar René Ornelis
Sección:

Guatemala, noviembre de 2026

Contenido

Contenido 1
1. Descripción general 3
1.1 Propósito del sistema 3
1.2 Objetivos 3
1.3 Alcance funcional 3
1.4 Entorno técnico 4
2. Modelo entidad-relación 5
2.1 Diagrama entidad-relación 5
2.2 Entidades 7
2.3 Relaciones 7
2.4 Decisiones de diseño 8
3. Diccionario de datos 8
3.10 Vistas, funciones y roles 12
4. Scripts de creación de la estructura (DDL) 12
5. Manual de configuración y activación de auditoría 13
5.1 Roles y permisos 13
5.2 Consultar el diccionario de datos 14
5.3 Activar la auditoría de una tabla desde SQL 14
5.4 Activar la auditoría desde la interfaz web 15
5.5 Desactivar la auditoría 15
5.6 Verificar el estado real de los triggers 15
5.7 Identificar la estación de trabajo 16
5.8 Validaciones que aplica el sistema 16
6. Procedimiento de instalación y despliegue de triggers 16
6.1 Requisitos previos 16
6.2 Instalación de PostgreSQL 16
6.3 Instalación de AutoAudit 17
6.4 Principios de diseño de los triggers 18
6.5 La función genérica fn_audit_generic 18
6.6 El generador dinámico de triggers 18
6.7 Ejemplo de trigger generado 19
6.8 Desinstalación 19
6.9 Manejo de errores 19
6.10 Lista de verificación del despliegue 20
7. Consultas históricas 20
7.1 Ejemplos de uso 20

7.2 Exportación de reportes 21
7.3 Consulta directa sobre la bitácora 21
8. Interfaz web 21
8.1 Arquitectura 21
8.2 Ejecución 22
8.3 Pantallas 22
8.4 Seguridad de la interfaz 24
9. Seguridad y cumplimiento de restricciones 25
10. Pruebas de validación 25
10.1 Pruebas funcionales en SQL 26
10.2 Pruebas automáticas de la interfaz 27
10.3 Validación manual guiada 27
10.4 Casos de prueba exigidos por el enunciado 28
11. Pruebas de rendimiento 28
11.1 Objetivo y metodología 28
11.2 Entorno de prueba 28
11.3 Resultados 29
11.4 Tamaño de la bitácora 30
11.5 Consultas históricas con 7.7 millones de filas 30
11.6 Análisis 30
11.7 Recomendaciones de operación 31
12. Verificación de los criterios de aceptación 31
13. Limitaciones y trabajo futuro 31
13.1 Limitaciones conocidas 31
13.2 Trabajo futuro 32
14. Conclusiones 32
Anexo A. Función genérica fn_audit_generic 33
Anexo B. Repositorio y estructura del proyecto 35
B.1 Resumen de comandos 35

1. Descripción general

1.1 Propósito del sistema

AutoAudit es un sistema de auditoría configurable para bases de datos PostgreSQL. Permite que un administrador elija qué tablas y columnas deben vigilarse y para cuáles eventos (INSERT, UPDATE y DELETE). A partir de esa configuración, el sistema genera automáticamente los triggers necesarios y registra cada cambio en una bitácora centralizada, con los valores anteriores y nuevos, el usuario responsable, la estación de trabajo, la dirección IP y la marca temporal.

La solución se compone de tres partes: (a) un esquema de base de datos (`autoaudit`) con tablas de configuración, registro de triggers y bitácora; (b) un conjunto de funciones PL/pgSQL que validan la configuración, generan e instalan los triggers y ofrecen consultas históricas; y (c) una interfaz web desarrollada en Python (Flask) para configurar la auditoría, consultar el historial y exportar reportes.

1.2 Objetivos

Objetivo principal. Desarrollar una solución de auditoría automática que capture cambios en tablas configurables mediante la generación dinámica de triggers, permitiendo la consulta histórica de modificaciones por usuario o por columna específica.

Tabla 1. *Cumplimiento de los objetivos específicos del proyecto.*

Objetivo específico	Cómo se cumple	Sección
Diseñar el modelo entidad-relación completo	Modelo de 9 entidades con diagrama, descripción de relaciones y diccionario de datos.	2 y 3
Configurar tablas y columnas a auditar a partir del diccionario de datos	Vistas `v_db_tables` y `v_db_columns` sobre `pg_catalog`; funciones de registro que validan existencia; pantalla de configuración web.	5 y 8
Generar automáticamente triggers para INSERT, UPDATE y DELETE	Generador `install_triggers`: un trigger AFTER por tabla y evento habilitado, registrado en `trigger_registry`.	6
Crear tablas de bitácora con información completa	`audit_log` con valores antiguos/nuevos, usuario, estación, IP, marca temporal, PK y configuración usada.	3 y 4
Interfaces de consulta histórica por usuario o columna	Funciones `history_by_*` y pantalla web de historial con filtros y exportación CSV/JSON.	7 y 8

1.3 Alcance funcional

Módulo	Funcionalidad implementada
Configuración de auditoría	Selección de tablas existentes según el diccionario de datos; selección granular de columnas; activación o desactivación por tipo de operación; persistencia en tablas especializadas.
Generación de triggers	Procesamiento automático de la configuración; generación del SQL de cada trigger; triggers AFTER por fila; registro y verificación del estado real de cada trigger.
Almacenamiento de bitácora	Tabla centralizada de solo-append; una fila por columna modificada; valores antiguos y nuevos; contexto de la operación (usuario, estación, IP, fecha y hora, sesión).
Consulta y reporte	Búsqueda por usuario, por columna y por fila; filtro por rango de fechas; orden cronológico; exportación a CSV y JSON; vista de impresión.

1.4 Entorno técnico

Componente	Especificación
Gestor de base de datos	PostgreSQL 18 (requisito del proyecto). Requiere PostgreSQL 15 o superior por el uso de `UNIQUE NULLS NOT DISTINCT`.
Sistema operativo objetivo	Ubuntu 26.04 LTS (requisito del proyecto). El desarrollo y las pruebas de rendimiento se ejecutaron en Windows con PostgreSQL 18.6; los scripts son independientes del sistema operativo.
Lenguaje de los triggers	PL/pgSQL (lenguaje procedural de PostgreSQL).
Servidor e interfaz	Python 3.10+ con Flask 3 y psycopg 3 (decisión de la clase: aplicación web sencilla).
Cliente de consola	`psql` para instalar los scripts y ejecutar las pruebas.

2. Modelo entidad-relación

2.1 Diagrama entidad-relación

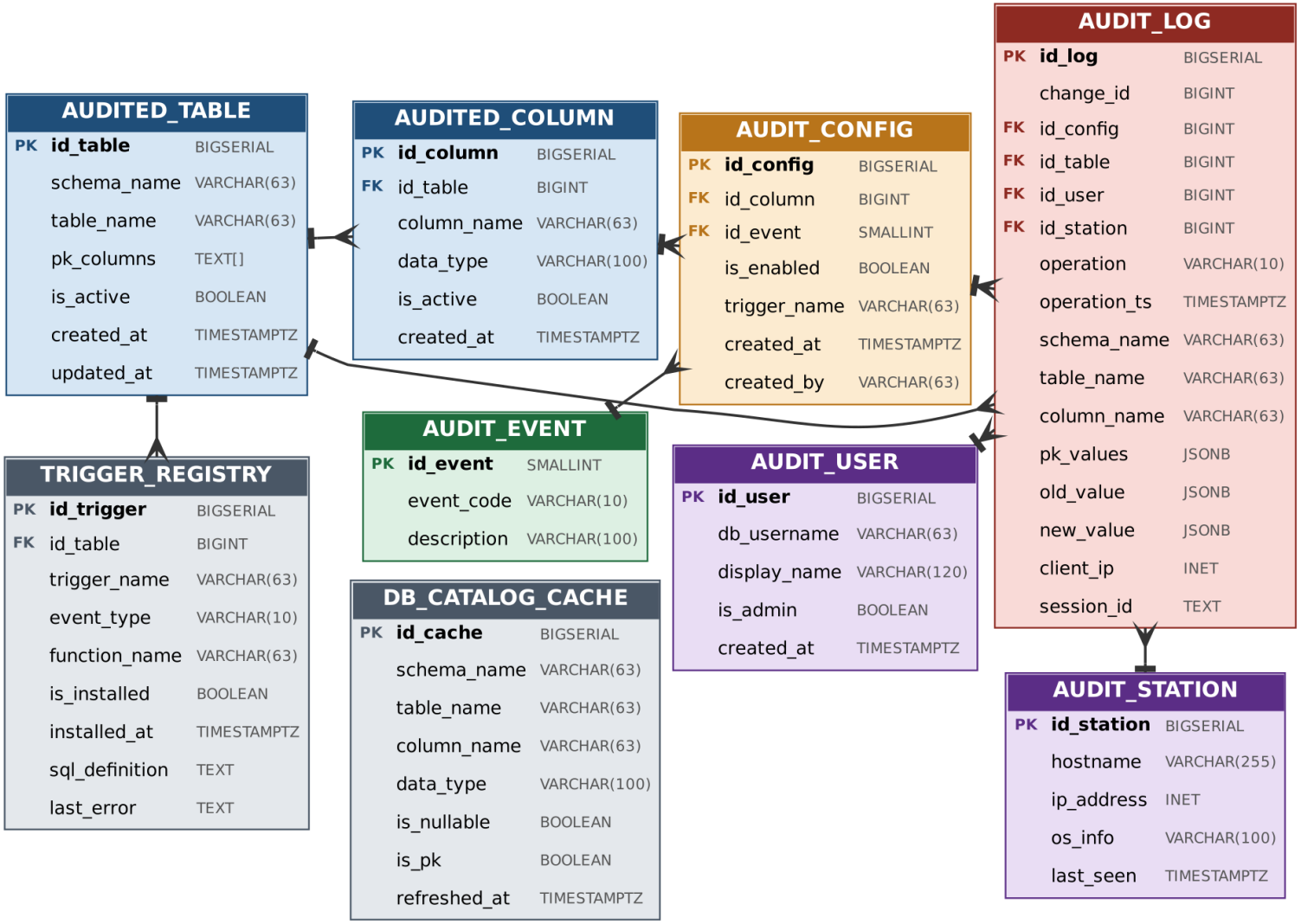


Figura 1. *Diagrama entidad-relación de AutoAudit. Notación de pata de gallo: el extremo con una raya es el lado «uno» y el extremo con tres líneas es el lado «muchos». Colores: azul = tablas y columnas auditadas; verde = catálogo de eventos; naranja = configuración; rojo = bitácora; morado = usuarios y estaciones; gris = registro de triggers y caché del catálogo.*

2.2 Entidades

El modelo separa cuatro responsabilidades: (a) metadatos de configuración, (b) registro de triggers instalados, (c) bitácora de cambios y (d) actores (usuarios y estaciones de trabajo). Se añade una caché opcional del diccionario de datos.

Entidad	Descripción
AUDITED_TABLE	Tabla de la base de datos origen que se desea auditar (esquema + nombre). Guarda las columnas de su clave primaria (pk_columns) para identificar la fila afectada. is_active permite desactivar la auditoría sin perder el historial.
AUDITED_COLUMN	Columna específica de una tabla auditada. data_type se toma del catálogo de PostgreSQL al momento de registrarla.
AUDIT_EVENT	Catálogo de eventos: INSERT, UPDATE y DELETE. Permite activar la auditoría por tipo de operación de forma independiente.
AUDIT_CONFIG	Configuración concreta: qué columna y qué evento se auditan. Es la entidad central de configuración; guarda el nombre del trigger generado y quién la activó.
TRIGGER_REGISTRY	Registro de los triggers realmente instalados (a lo sumo uno por tabla y evento): nombre, evento, función, DDL ejecutado, fecha de instalación y último error.
AUDIT_LOG	Bitácora centralizada. Cada fila es un cambio capturado en una columna. Preserva los datos aunque se elimine el registro de negocio o la configuración.
AUDIT_USER	Usuario de base de datos que realizó el cambio (o administró el sistema). is_admin identifica a los administradores.
AUDIT_STATION	Estación de trabajo desde la que se originó la sesión (nombre de host e IP).
DB_CATALOG_CACHE	Caché opcional del diccionario de datos (pg_catalog) para no consultar el catálogo en cada refresco de la interfaz. No tiene claves foráneas.

2.3 Relaciones

Tabla 2. *Relaciones del modelo (todas de uno a muchos).*

Relación	Significado	Acción al borrar
AUDITED_TABLE 1—N AUDITED_COLUMN	Una tabla tiene muchas columnas auditables.	CASCADE
AUDITED_COLUMN 1—N AUDIT_CONFIG	Una columna puede configurarse para varios eventos.	CASCADE
AUDIT_EVENT 1—N AUDIT_CONFIG	Un evento se usa en muchas configuraciones.	NO ACTION
AUDITED_TABLE 1—N TRIGGER_REGISTRY	Una tabla tiene un trigger por evento habilitado.	CASCADE
AUDIT_CONFIG 1—N AUDIT_LOG	Cada fila de bitácora referencia la configuración que la originó.	SET NULL
AUDITED_TABLE 1—N AUDIT_LOG	Cada fila de bitácora referencia la tabla configurada de origen (integridad referencial con las tablas origen configuradas).	SET NULL
AUDIT_USER 1—N AUDIT_LOG	Quién realizó el cambio.	NO ACTION
AUDIT_STATION 1—N AUDIT_LOG	Desde qué estación se realizó el cambio.	NO ACTION

2.4 Decisiones de diseño

- **Una fila de bitácora por columna modificada.** Cada fila apunta a su propia configuración (`id_config`), tal como exige el enunciado (cada registro referencia una configuración específica). Las filas que pertenecen a una misma operación sobre la fila de negocio se agrupan con `change_id`. Esto también simplifica la consulta por columna.
- **Preservación de datos.** `audit_log` no tiene claves foráneas hacia las tablas de negocio. Guarda `schema_name`, `table_name`, `column_name` y `pk_values` de forma independiente, de modo que el historial sobrevive aunque se borre la fila origen. Las claves `id_config` e `id_table` pasan a NULL si se elimina la configuración (`ON DELETE SET NULL`).
- **Valores en JSONB.** `old_value` y `new_value` son JSONB escalares: aceptan cualquier tipo de columna sin esquema rígido y conservan la precisión de los números (por ejemplo `1800.00`).
- **Solo se guardan las columnas auditadas.** La clave primaria se almacena aparte en `pk_values`; el resto de la fila nunca se copia a la bitácora (privacidad y tamaño).
- **Usuario real.** Se registra `session_user`, no `current_user`: dentro de una función `SECURITY DEFINER`, `current_user` sería el dueño de la función y no quien hizo el cambio.
- **Bitácora de solo-append.** Un trigger de protección bloquea UPDATE, DELETE y TRUNCATE sobre `audit_log`, incluso para su dueño.

3. Diccionario de datos

Todas las tablas del sistema residen en el esquema `autoaudit`, para no mezclarse con las tablas de negocio. Las columnas de texto usan la longitud máxima de un identificador de PostgreSQL (63 caracteres) donde corresponde. Las marcas temporales son `TIMESTAMPTZ`.

3.1 AUDIT_EVENT

Catálogo de eventos auditables.

Columna	Tipo	Nulo	Clave / valor	Descripción
`id_event`	SMALLINT	NO	PK	1 = INSERT, 2 = UPDATE, 3 = DELETE
`event_code`	VARCHAR(10)	NO	UNIQUE, CHECK	INSERT, UPDATE o DELETE
`description`	VARCHAR(100)	SÍ		Descripción legible del evento

3.2 AUDITED_TABLE

Tablas de negocio registradas para auditoría. Restricción `UNIQUE (schema_name, table_name)`.

Columna	Tipo	Nulo	Clave / valor	Descripción
`id_table`	BIGSERIAL	NO	PK	Identificador interno de la tabla auditada
`schema_name`	VARCHAR(63)	NO		Esquema PostgreSQL (por ejemplo `public`)
`table_name`	VARCHAR(63)	NO		Nombre de la tabla origen
`pk_columns`	TEXT[]	NO	DEFAULT '{}'	Columnas de la clave primaria de la tabla origen (se calculan al registrarla)
`is_active`	BOOLEAN	NO	DEFAULT TRUE	TRUE = auditoría activa para la tabla

Columna	Tipo	Nulo	Clave / valor	Descripción
created_at	TIMESTAMPTZ	NO	DEFAULT now()	Fecha de registro en el sistema
updated_at	TIMESTAMPTZ	SÍ		Última modificación de la configuración

3.3 AUDITED_COLUMN

Columnas vigiladas. Restricción `UNIQUE (id_table, column_name)`.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_column	BIGSERIAL	NO	PK	Identificador de la columna auditada
id_table	BIGINT	NO	FK → AUDITED_TABLE	Tabla a la que pertenece (`ON DELETE CASCADE`)
column_name	VARCHAR(63)	NO		Nombre de la columna en PostgreSQL
data_type	VARCHAR(100)	NO		Tipo de dato obtenido del catálogo (por ejemplo `numeric(12,2)`)
is_active	BOOLEAN	NO	DEFAULT TRUE	TRUE = se audita esta columna
created_at	TIMESTAMPTZ	NO	DEFAULT now()	Fecha de alta

3.4 AUDIT_CONFIG

Pares columna × evento. Restricción `UNIQUE (id_column, id_event)`.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_config	BIGSERIAL	NO	PK	Identificador de la configuración
id_column	BIGINT	NO	FK → AUDITED_COLUMN	Columna que se audita (`ON DELETE CASCADE`)
id_event	SMALLINT	NO	FK → AUDIT_EVENT	Evento que se captura
is_enabled	BOOLEAN	NO	DEFAULT TRUE	TRUE = trigger activo para este par
trigger_name	VARCHAR(63)	SÍ		Nombre del trigger que lo atiende; lo llena el generador
created_at	TIMESTAMPTZ	NO	DEFAULT now()	Cuándo se creó la configuración
created_by	VARCHAR(63)	SÍ		Usuario administrador que la activó (`session_user`)

3.5 TRIGGER_REGISTRY

Triggers realmente instalados. Restricciones `UNIQUE (id_table, event_type)` y `UNIQUE (id_table, trigger_name)`: como máximo un trigger por tabla y evento.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_trigger	BIGSERIAL	NO	PK	Identificador del trigger
id_table	BIGINT	NO	FK → AUDITED_TABLE	Tabla sobre la que actúa (`ON DELETE CASCADE`)
trigger_name	VARCHAR(63)	NO		Nombre en PostgreSQL (`trg_audit_<tabla>_ins/upd/del`)
event_type	VARCHAR(10)	NO		INSERT, UPDATE o DELETE

Columna	Tipo	Nulo	Clave / valor	Descripción
function_name	VARCHAR(63)	NO		Función asociada (autoaudit.fn_audit_generic)
is_installed	BOOLEAN	NO	DEFAULT FALSE	TRUE si el trigger fue instalado
installed_at	TIMESTAMPTZ	SÍ		Fecha del último CREATE TRIGGER
sql_definition	TEXT	SÍ		DDL ejecutado para crear el trigger
last_error	TEXT	SÍ		Último error al generar o instalar (NULL si todo salió bien)

3.6 AUDIT_USER

Usuarios de base de datos que operan o administran.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_user	BIGSERIAL	NO	PK	Identificador interno
db_username	VARCHAR(63)	NO	UNIQUE	Nombre de rol o usuario en PostgreSQL
display_name	VARCHAR(120)	SÍ		Nombre legible
is_admin	BOOLEAN	NO	DEFAULT FALSE	TRUE = administrador de auditoría
created_at	TIMESTAMPTZ	NO	DEFAULT now()	Alta del usuario en el sistema

3.7 AUDIT_STATION

Estaciones de trabajo. Restricción `UNIQUE NULLS NOT DISTINCT (hostname, ip_address)`.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_station	BIGSERIAL	NO	PK	Identificador de estación
hostname	VARCHAR(255)	SÍ		Nombre de host (lo envía la aplicación cliente)
ip_address	INET	SÍ		Dirección IP del cliente
os_info	VARCHAR(100)	SÍ		Sistema operativo (lo envía la aplicación cliente)
last_seen	TIMESTAMPTZ	SÍ		Última actividad registrada (se actualiza como máximo 1 vez por minuto)

3.8 AUDIT_LOG

Bitácora centralizada. Una fila por columna auditada que cambió.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_log	BIGSERIAL	NO	PK	Identificador del registro de bitácora
change_id	BIGINT	NO		Agrupa las filas de una misma operación sobre la fila de negocio
id_config	BIGINT	SÍ	FK → AUDIT_CONFIG	Configuración que originó el registro (ON DELETE SET NULL)
id_table	BIGINT	SÍ	FK → AUDITED_TABLE	Tabla configurada de origen (ON DELETE SET NULL)
id_user	BIGINT	SÍ	FK → AUDIT_USER	Usuario de base de datos que ejecutó la operación

Columna	Tipo	Nulo	Clave / valor	Descripción
id_station	BIGINT	SÍ	FK → AUDIT_STATION	Estación de trabajo de origen
operation	VARCHAR(10)	NO	CHECK	INSERT, UPDATE o DELETE
operation_ts	TIMESTAMPTZ	NO	DEFAULT clock_timestamp()	Marca temporal del cambio
schema_name	VARCHAR(63)	NO		Esquema de la tabla origen (desnormalizado para preservar)
table_name	VARCHAR(63)	NO		Tabla origen (desnormalizado)
column_name	VARCHAR(63)	NO		Columna modificada (desnormalizado)
pk_values	JSONB	SÍ		Valores de la PK de la fila afectada, por ejemplo `{"id": 1}`
old_value	JSONB	SÍ		Valor anterior de la columna (NULL en INSERT)
new_value	JSONB	SÍ		Valor nuevo de la columna (NULL en DELETE)
client_ip	INET	SÍ		IP del cliente (`inet_client_addr()`); NULL si la conexión es por socket local
session_id	TEXT	SÍ		Identificador de sesión: `<pid del backend>/<id de transacción>`

Índices. `idx_audit_log_ts (operation_ts DESC)`, `idx_audit_log_user (id_user, operation_ts DESC)`, `idx_audit_log_column (schema_name, table_name, column_name, operation_ts DESC)` e `idx_audit_log_change (change_id)`. Se evitó el índice GIN sobre los valores porque encarece cada escritura (ver sección 11). Existe un índice opcional sobre `pk_values`, comentado en el script.

Protección. Los triggers `trg_audit_log_immutable` (UPDATE/DELETE) y `trg_audit_log_no_truncate` (TRUNCATE) impiden modificar la bitácora. La única excepción es la acción `ON DELETE SET NULL` de las claves foráneas, que solo pone en NULL `id_config` e `id_table`.

3.9 DB_CATALOG_CACHE

Caché del diccionario de datos. Restricción `UNIQUE (schema_name, table_name, column_name)`.

Columna	Tipo	Nulo	Clave / valor	Descripción
id_cache	BIGSERIAL	NO	PK	Identificador de caché
schema_name	VARCHAR(63)	NO		Esquema
table_name	VARCHAR(63)	NO		Tabla
column_name	VARCHAR(63)	NO		Columna
data_type	VARCHAR(100)	NO		Tipo de dato
is_nullable	BOOLEAN	NO		Permite NULL
is_pk	BOOLEAN	NO	DEFAULT FALSE	Forma parte de la clave primaria
refreshed_at	TIMESTAMPTZ	NO	DEFAULT now()	Última sincronización con `pg_catalog`

3.10 Vistas, funciones y roles

Tabla 3. *Objetos programáticos del esquema autoaudit.*

Objeto	Tipo	Propósito	Acceso
`v_db_tables`	Vista	Tablas de usuario existentes en la base (excluye esquemas del sistema y `autoaudit`).	Admin
`v_db_columns`	Vista	Columnas con tipo, nulabilidad y marca de clave primaria.	Admin
`refresh_catalog_cache(esquema)`	Función	Sincroniza `db_catalog_cache` con el catálogo.	Admin
`register_table`, `register_column`	Funciones	Registran tabla y columnas validando su existencia en el catálogo.	Admin
`set_audit_config`	Función	Habilita o inhabilita un par columna × evento.	Admin
`enable_audit`, `disable_audit`	Funciones	Atajos: registran, configuran e instalan; o desactivan y reajustan los triggers.	Admin
`install_triggers`, `uninstall_triggers`	Funciones	Generador dinámico de triggers y su desinstalación.	Admin
`v_trigger_status`	Vista	Compara `trigger_registry` con `pg_trigger` (estado real).	Admin
`fn_audit_generic`	Función trigger	Función que ejecutan todos los triggers de auditoría.	Interna
`resolve_user`, `resolve_station`, `assert_admin`	Funciones	Resuelven usuario y estación; verifican privilegios de administrador.	Interna
`fn_audit_log_immutable`	Función trigger	Bloquea UPDATE, DELETE y TRUNCATE sobre la bitácora.	Interna
`v_audit_log_detail`	Vista	Bitácora unida con usuario y estación.	Lector
`history_by_user`, `history_by_column`, `history_by_row`	Funciones	Consultas históricas parametrizadas.	Lector

Rol	Privilegios
`autoaudit_admin`	Ejecuta las funciones de configuración y lee las tablas de configuración. Hereda `autoaudit_reader`. No tiene permiso de escritura directa sobre ninguna tabla.
`autoaudit_reader`	Lee `audit_log`, `audit_user`, `audit_station` y `v_audit_log_detail`; ejecuta las consultas históricas.
`PUBLIC`	Sin acceso al esquema `autoaudit`.

4. Scripts de creación de la estructura (DDL)

La instalación se divide en ocho scripts numerados que se ejecutan en orden mediante `install.sql`. Todos son idempotentes (`IF NOT EXISTS`, `CREATE OR REPLACE`), por lo que pueden ejecutarse más de una vez sin errores.

Tabla 4. *Scripts de instalación (carpeta `sql/`).*

Script	Contenido
`01_schema.sql`	Esquema `autoaudit`, tablas, restricciones, índices, secuencia `seq_change_id` y triggers de protección de la bitácora.

Script	Contenido
02_security.sql	Roles autoaudit_admin y autoaudit_reader; función assert_admin().
03_catalog.sql	Consultas al diccionario de datos: v_db_tables, v_db_columns, refresh_catalog_cache().
04_config.sql	register_table, register_column, set_audit_config, enable_audit, disable_audit.
05_trigger_function.sql	fn_audit_generic() y las funciones auxiliares resolve_user() y resolve_station().
06_trigger_generator.sql	Generador dinámico: install_triggers(), uninstall_triggers() y la vista v_trigger_status.
07_queries.sql	Vista v_audit_log_detail y consultas históricas history_by_user, history_by_column, history_by_row.
08_grants.sql	Permisos: revoca todo a PUBLIC y otorga a los roles de AutoAudit.
90_demo.sql, 91_usuarios_demo.sql	Opcionales: ejemplo de uso y usuarios de demostración. No forman parte de la instalación.

A modo de ejemplo se reproduce la definición de la tabla central de la bitácora, tal como aparece en 01_schema.sql:

```
CREATE TABLE IF NOT EXISTS autoaudit.audit_log (
    id_log       BIGSERIAL    PRIMARY KEY,
    change_id    BIGINT       NOT NULL,
    id_config    BIGINT       REFERENCES autoaudit.audit_config(id_config) ON DELETE SET NULL,
    id_table     BIGINT       REFERENCES autoaudit.audited_table(id_table) ON DELETE SET NULL,
    id_user      BIGINT       REFERENCES autoaudit.audit_user(id_user),
    id_station   BIGINT       REFERENCES autoaudit.audit_station(id_station),
    operation    VARCHAR(10)  NOT NULL CHECK (operation IN ('INSERT','UPDATE','DELETE')),
    operation_ts TIMESTAMPTZ  NOT NULL DEFAULT clock_timestamp(),
    schema_name  VARCHAR(63)  NOT NULL,
    table_name   VARCHAR(63)  NOT NULL,
    column_name  VARCHAR(63)  NOT NULL,
    pk_values    JSONB,
    old_value    JSONB,
    new_value    JSONB,
    client_ip    INET,
    session_id   TEXT
);
```

El resto de las definiciones se encuentra en el repositorio del proyecto (anexo B) y su estructura está descrita campo por campo en el diccionario de datos (sección 3).

5. Manual de configuración y activación de auditoría

5.1 Roles y permisos

Solo los usuarios con privilegios administrativos pueden configurar la auditoría. En AutoAudit esto se resuelve con roles de PostgreSQL: las funciones de configuración verifican que quien las invoca sea superusuario o miembro de autoaudit_admin; de lo contrario lanzan un error de privilegios insuficientes. Para otorgar o quitar acceso:

```
-- Dar permisos de administrador (configurar) o de lector (solo consultar)
GRANT autoaudit_admin  TO mi_usuario;
GRANT autoaudit_reader TO otro_usuario;

-- Quitar permisos
```

```
REVOKE autoaudit_admin FROM mi_usuario;
```

Los usuarios de ejemplo que acompañan al proyecto (`sql/91_usuarios_demo.sql`) son `admin_demo`, `lector_demo` y `usuario_demo`. Sus contraseñas son solo para demostración y deben cambiarse en cualquier otro entorno.

5.2 Consultar el diccionario de datos

Para listar las tablas y columnas disponibles se consulta el catálogo del gestor. El sistema las expone mediante dos vistas, que son las que usa la interfaz web:

```
-- Tablas de usuario disponibles para auditar
SELECT schema_name, table_name FROM autoaudit.v_db_tables ORDER BY 1, 2;

-- Columnas de una tabla (con tipo, nulabilidad y si forman parte de la PK)
SELECT column_name, data_type, is_nullable, is_pk
FROM autoaudit.v_db_columns
WHERE schema_name = 'public' AND table_name = 'clientes'
ORDER BY ordinal_position;
```

Las vistas se construyen sobre `pg_catalog`. Como referencia, las consultas equivalentes sobre `information_schema` son:

```
-- Tablas de un esquema
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema NOT IN ('pg_catalog','information_schema','autoaudit')
  AND table_type = 'BASE TABLE'
ORDER BY 1, 2;

-- Columnas de una tabla
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'clientes'
ORDER BY ordinal_position;
```

Para evitar consultar el catálogo en cada refresco de la interfaz puede poblarse la caché con `SELECT autoaudit.refresh_catalog_cache();` (opcionalmente indicando un esquema).

5.3 Activar la auditoría de una tabla desde SQL

Hay dos formas. La primera, `enable_audit`, hace todo en una sola llamada: registra la tabla, registra las columnas, activa los eventos e instala los triggers. Debe ejecutarla un administrador.

```
-- Auditar nombre, email y saldo de public.clientes en INSERT, UPDATE y DELETE
SELECT * FROM autoaudit.enable_audit('public', 'clientes',
                                     ARRAY['nombre','email','saldo']);

-- Solo algunos eventos:
SELECT * FROM autoaudit.enable_audit('public', 'clientes',
                                     ARRAY['saldo'], ARRAY['UPDATE','DELETE']);

-- Todas las columnas, todos los eventos:
SELECT * FROM autoaudit.enable_audit('public', 'clientes');
```

La segunda forma es paso a paso, útil para ver qué valida cada función:

1. `SELECT autoaudit.register_table('public', 'clientes');` valida que la tabla exista, detecta su clave primaria y la registra.
2. `SELECT autoaudit.register_column('public', 'clientes', 'saldo');` valida que la columna exista y guarda su tipo de dato.

3. `SELECT autoaudit.set_audit_config('public', 'clientes', 'saldo', 'UPDATE', TRUE);` habilita el par columna × evento.
4. `SELECT * FROM autoaudit.install_triggers('public', 'clientes');` genera e instala los triggers según la configuración habilitada.

Cada llamada a `enable_audit` o `install_triggers` devuelve una fila por evento con el nombre del trigger, el evento, el resultado (`INSTALLED`, `REMOVED`, `SKIPPED` o `ERROR`) y el detalle.

5.4 Activar la auditoría desde la interfaz web

1. Inicie sesión con un usuario administrador y abra **Configuración**. Se listan las tablas de usuario encontradas en el diccionario de datos, con su estado de auditoría.
2. Elija una tabla con **Configurar**. Se muestran sus columnas, su tipo de dato y una marca de clave primaria, con una casilla por cada evento (INSERT, UPDATE, DELETE).
3. Marque las columnas y los eventos deseados. El enlace «todos / ninguno» de cada evento marca o desmarca la columna completa.
4. Pulse **Guardar y aplicar**. El sistema valida la selección contra el catálogo, guarda la configuración e instala o elimina los triggers necesarios. Aparece el resultado de cada evento.
5. Verifique en la sección **Estado de los triggers** que todos figuren como «Sincronizado: sí».

5.5 Desactivar la auditoría

Alcance	Instrucción	Efecto
Tabla completa	`SELECT * FROM autoaudit.disable_audit('public', 'clientes');`	Marca la tabla como inactiva y elimina todos sus triggers.
Una columna	`... disable_audit('public', 'clientes', ARRAY['saldo']);`	Inhabilita la columna en todos los eventos y reajusta los triggers.
Un evento	`... disable_audit('public', 'clientes', NULL, ARRAY['DELETE']);`	Inhabilita el evento para todas las columnas y elimina su trigger.

En ningún caso se borran registros de `audit_log`: la preservación histórica es un requisito del sistema. Para reactivar basta con volver a ejecutar `enable_audit`.

5.6 Verificar el estado real de los triggers

La configuración debe reflejar el estado actual de la auditoría. La vista `v_trigger_status` compara lo que dice `trigger_registry` con lo que existe realmente en PostgreSQL (`pg_trigger`):

```
SELECT schema_name, table_name, event_type, trigger_name,
       registry_says_installed, exists_in_postgres, in_sync, last_error
FROM autoaudit.v_trigger_status
ORDER BY 1, 2, 3;
```

Si alguien elimina un trigger manualmente, `in_sync` será FALSE; al volver a ejecutar `install_triggers` (o «Guardar y aplicar» en la interfaz) el estado se corrige.

5.7 Identificar la estación de trabajo

El servidor conoce la IP del cliente, pero no su nombre de host ni su sistema operativo. La aplicación cliente puede informarlos al abrir la conexión:

```
SET autoaudit.hostname = 'PC-CONTABILIDAD-01';
SET autoaudit.os_info  = 'Windows 11';
```

> **Nota sobre la IP**
> Si el cliente se conecta por socket local de Unix, `inet_client_addr()` devuelve NULL y la IP queda vacía. Para registrarla, conéctese por TCP (por ejemplo `psql -h 127.0.0.1 ...`).

5.8 Validaciones que aplica el sistema

Validación	Dónde se aplica	Resultado si falla
La tabla existe en la base de datos	`register_table`, `install_triggers`	Error «la tabla … no existe o no es auditable»
La columna existe en la tabla	`register_column`	Error «la columna … no existe»
El evento es INSERT, UPDATE o DELETE	`set_audit_config`, restricción CHECK	Error «evento inválido»
La tabla y la columna están registradas antes de configurarlas	`register_column`, `set_audit_config`	Error indicando qué función usar primero
Quien configura es administrador	`assert_admin()` en cada función de configuración	Error de privilegios insuficientes
La tabla tiene clave primaria	`register_table`	Advertencia: `pk_values` quedará vacío

6. Procedimiento de instalación y despliegue de triggers

6.1 Requisitos previos

- PostgreSQL 15 o superior (objetivo del proyecto: PostgreSQL 18) con acceso de superusuario para la instalación.
- El cliente de consola `psql` y la carpeta del proyecto con los scripts de `sql/`.
- Para la interfaz web: Python 3.10 o superior.
- El usuario que instala será el dueño de las funciones. Para crear los triggers debe ser superusuario o dueño de las tablas que se auditen.

6.2 Instalación de PostgreSQL

Windows. Descargue el instalador desde postgresql.org, conserve el puerto 5432 y anote la contraseña del usuario `postgres`. Para usar `psql` en una ventana de `cmd`, agregue su carpeta al `PATH` de esa sesión:

```
set PATH=C:\Program Files\PostgreSQL\18\bin;%PATH%
set PGCLIENTENCODING=UTF8
psql --version
```

Ubuntu 26.04 LTS. Instale el paquete del gestor y compruebe la versión. Si el repositorio de la distribución no ofrece la versión 18, utilice el repositorio oficial PGDG:

```
sudo apt update
sudo apt install postgresql
psql --version

# Si hace falta la versión 18 desde PGDG:
sudo apt install postgresql-common
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh
sudo apt install postgresql-18
```

6.3 Instalación de AutoAudit

Los comandos se ejecutan desde la carpeta del proyecto. Primero se crea la base de datos y luego se ejecuta el instalador, que corre los ocho scripts en orden:

```
:: Windows (cmd)
psql -U postgres -c "CREATE DATABASE autoaudit_db;"
psql -U postgres -d autoaudit_db -f sql\install.sql

# Ubuntu (bash)
sudo -u postgres createdb autoaudit_db
sudo -u postgres psql -d autoaudit_db -f sql/install.sql
```

La instalación termina con el mensaje `AutoAudit instalado correctamente.` Si se produce un error, el instalador se detiene en ese punto (`ON_ERROR_STOP`) para no dejar una instalación a medias. En Ubuntu, si el usuario `postgres` no puede leer la carpeta del proyecto, copie los scripts a un directorio legible (por ejemplo `/tmp`) o ejecute `psql -h localhost -U postgres`.

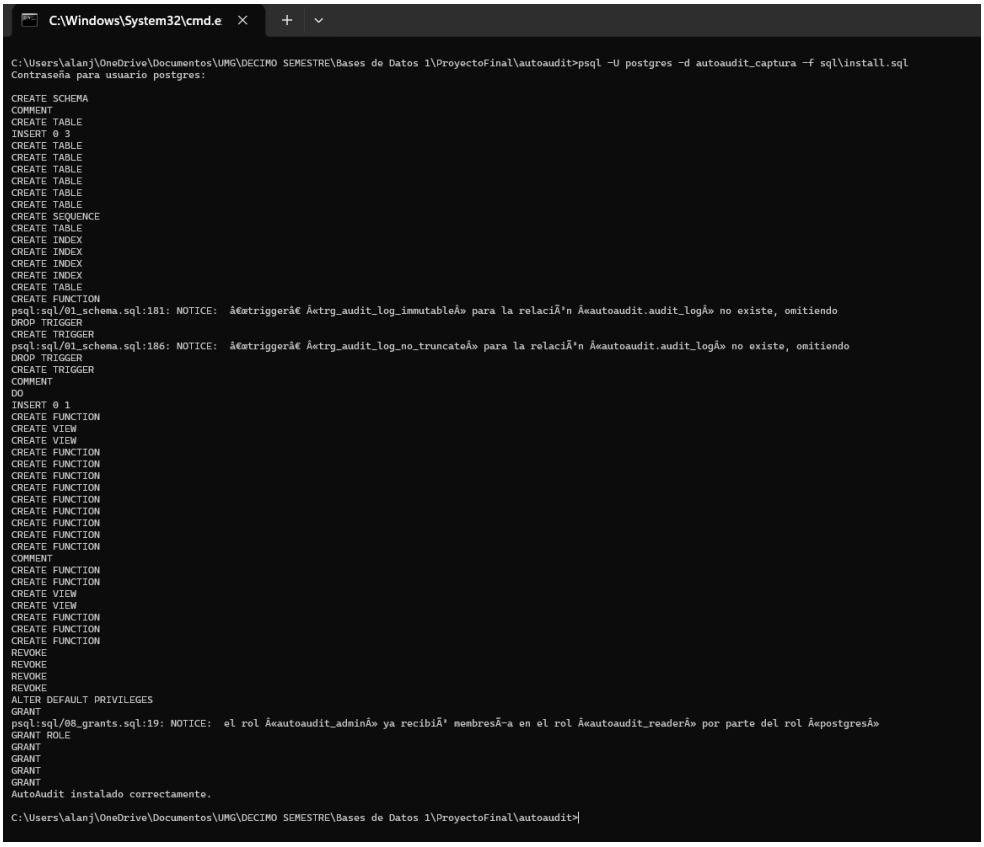


Figura 2. *Salida de la instalación: «AutoAudit instalado correctamente.»*

6.4 Principios de diseño de los triggers

- **Un trigger AFTER por tabla y por evento** (INSERT, UPDATE, DELETE), `FOR EACH ROW`, que ejecuta siempre la misma función genérica. Cada trigger atiende exclusivamente a una tabla origen.
- La función lee la configuración activa de `audit_config` para saber qué columnas registrar: no hay código duplicado ni SQL generado por tabla dentro de la función.
- Se almacenan solo las columnas configuradas, no la fila completa. En UPDATE se registran únicamente las que cambiaron de valor.
- Se capturan `session_user`, `inet_client_addr()`, `clock_timestamp()`, la estación y la clave primaria de la fila.
- La bitácora no tiene claves foráneas hacia las tablas de negocio: los datos sobreviven aunque se borre el registro de origen.
- Si no es posible escribir la bitácora, el error se propaga y se revierte el cambio original. Así nunca existen modificaciones sin su registro de auditoría.

6.5 La función genérica fn_audit_generic

La función se ejecuta como `SECURITY DEFINER` con un `search_path` fijo (`pg_catalog, autoaudit, pg_temp`), de modo que los usuarios de negocio no necesitan permisos sobre las tablas de AutoAudit. Su lógica, que se reproduce completa en el anexo A, sigue estos pasos:

1. Busca la tabla en `audited_table`; si no está registrada o no está activa, termina sin hacer nada.
2. Convierte la fila anterior (`OLD`) y la nueva (`NEW`) a JSONB, según el evento.
3. Consulta `audit_config` y obtiene las columnas habilitadas para el evento. En UPDATE descarta las columnas cuyo valor no cambió; si ninguna cambió, termina sin escribir.
4. Calcula `pk_values` con las columnas de clave primaria (valores anteriores en UPDATE y DELETE; nuevos en INSERT).
5. Resuelve el usuario (`session_user`) y la estación (IP y datos enviados por el cliente), creándolos si no existen.
6. Inserta en `audit_log`, en una sola sentencia, una fila por cada columna a registrar, con el mismo `change_id`.

6.6 El generador dinámico de triggers

`install_triggers(esquema, tabla)` implementa el módulo de generación. Para cada evento del catálogo (`audit_event`):

1. Verifica que quien invoca sea administrador y que la tabla esté registrada, activa y exista en la base de datos.
2. Determina si el evento tiene al menos una columna activa con configuración habilitada.
3. Si la tiene: elimina el trigger previo del mismo nombre, construye el DDL con `format('CREATE TRIGGER %I AFTER %s ON %I.%I FOR EACH ROW EXECUTE FUNCTION autoaudit.fn_audit_generic()', ...)`, lo ejecuta y lo registra en `trigger_registry` (con

`sql_definition`, `installed_at` y `is_installed = TRUE`). También anota el nombre del trigger en `audit_config`.

4. Si no la tiene y existía un trigger instalado para ese evento: lo elimina (`DROP TRIGGER IF EXISTS`) y marca `is_installed = FALSE`.
5. Si ocurre un error en un evento, lo captura en un bloque de excepción, lo guarda en `trigger_registry.last_error` y continúa con los demás eventos.

El nombre de cada trigger se forma como `trg_audit_<tabla>_ins|upd|del` (la parte de la tabla se limita a 45 caracteres para respetar el máximo de 63 de PostgreSQL). Como `trigger_registry` es único por (tabla, evento), nunca hay más de un trigger de auditoría por tabla y evento.

Tabla 5. *Resultados que devuelve el generador.*

Estado	Significado
`INSTALLED`	El trigger fue creado (o recreado) y registrado.
`REMOVED`	El evento ya no tiene configuración habilitada y su trigger fue eliminado.
`SKIPPED`	El evento no tiene configuración ni trigger previo; no se hizo nada.
`ERROR`	Falló la creación o eliminación; el detalle queda en `last_error`.

6.7 Ejemplo de trigger generado

Para `public.clientes` con los tres eventos configurados, el generador ejecuta y registra:

```
CREATE TRIGGER trg_audit_clientes_ins AFTER INSERT ON public.clientes
    FOR EACH ROW EXECUTE FUNCTION autoaudit.fn_audit_generic();
CREATE TRIGGER trg_audit_clientes_upd AFTER UPDATE ON public.clientes
    FOR EACH ROW EXECUTE FUNCTION autoaudit.fn_audit_generic();
CREATE TRIGGER trg_audit_clientes_del AFTER DELETE ON public.clientes
    FOR EACH ROW EXECUTE FUNCTION autoaudit.fn_audit_generic();
```

y en `trigger_registry` queda una fila por evento con el DDL en `sql_definition`, `is_installed = TRUE` y `last_error` vacío.

6.8 Desinstalación

`uninstall_triggers(esquema, tabla)` elimina todos los triggers instalados de la tabla, los marca como no instalados en `trigger_registry` y limpia el nombre de trigger en `audit_config`. `disable_audit` invoca esta función cuando se desactiva la tabla completa. La bitácora no se toca.

6.9 Manejo de errores

Situación	Comportamiento
Falla un `CREATE TRIGGER` o un `DROP TRIGGER`	El error se captura por evento; el DDL fallido se revierte, `last_error` guarda el mensaje y el generador continúa con los demás eventos.
La tabla ya no existe al instalar triggers	Error claro («la tabla … ya no existe en la base de datos»); no se crea nada.
Un usuario sin rol administrador invoca una función de configuración	Error de privilegios insuficientes; no se modifica nada.
Falla la escritura en `audit_log` durante un trigger	El error se propaga y se revierte la operación de negocio completa (política de integridad: sin cambios sin auditoría).

Situación	Comportamiento
Alguien borra un trigger manualmente	`v_trigger_status` muestra «Sincronizado: NO»; se corrige reinstalando.

6.10 Lista de verificación del despliegue

1. Instalar PostgreSQL y crear la base de datos.
2. Ejecutar `sql/install.sql` y comprobar el mensaje final.
3. Asignar `autoaudit_admin` a los administradores y `autoaudit_reader` a quienes solo consulten.
4. Activar la auditoría de cada tabla con `enable_audit` o desde la interfaz web.
5. Comprobar con `v_trigger_status` que todos los triggers estén sincronizados.
6. Realizar un cambio de prueba y confirmarlo con `history_by_column` o en la pantalla de historial.

7. Consultas históricas

Las consultas históricas se implementan como funciones parametrizadas sobre la vista `v_audit_log_detail`, que une la bitácora con el usuario y la estación. Todas aceptan un rango de fechas (`p_from` inclusivo, `p_to` exclusivo), un límite de filas (NULL = sin límite) y el orden (`p_desc = TRUE`: más reciente primero; `FALSE`: cronológico ascendente). Al ser funciones con parámetros, no hay concatenación de texto y se evita la inyección SQL.

Tabla 6. *Funciones de consulta histórica.*

Función	Parámetros	Devuelve
`history_by_user`	`p_username`, `p_from`, `p_to`, `p_limit` (1000), `p_desc` (TRUE)	Todos los cambios realizados por un usuario.
`history_by_column`	`p_schema`, `p_table`, `p_column`, `p_from`, `p_to`, `p_limit`, `p_desc`	Todos los cambios de una columna específica.
`history_by_row`	`p_schema`, `p_table`, `p_pk` (JSONB), `p_from`, `p_to`, `p_limit`, `p_desc`	La historia de una fila identificada por su clave primaria.

Las tres devuelven las mismas columnas: `id_log`, `change_id`, `operation_ts`, `operation`, `db_username`, `hostname`, `client_ip`, `schema_name`, `table_name`, `column_name`, `pk_values`, `old_value`, `new_value`, `id_config` y `session_id`.

7.1 Ejemplos de uso

```
-- Por usuario (últimos 1000 cambios)
SELECT * FROM autoaudit.history_by_user('juan');

-- Por usuario, en un rango de fechas y en orden cronológico
SELECT * FROM autoaudit.history_by_user('juan', '2026-11-01', '2026-12-01', 500, FALSE);

-- Por columna específica
SELECT operation_ts, operation, db_username, old_value, new_value
FROM autoaudit.history_by_column('public', 'clientes', 'saldo', NULL, NULL, NULL, FALSE);
```

```
-- Historia de una fila (por su clave primaria)
SELECT * FROM autoaudit.history_by_row('public', 'clientes', '{"id": 1}');
```

Tabla 7. *Ejemplo de salida de la consulta por columna `saldo` (ejemplo ilustrativo).*

operation_ts	operation	db_username	old_value	new_value
2026-11-07 10:15:02	INSERT	postgres	—	1500.00
2026-11-07 10:16:40	UPDATE	postgres	1500.00	1800.00
2026-11-07 10:18:11	DELETE	postgres	1800.00	—

7.2 Exportación de reportes

Desde `psql` el resultado de cualquier consulta puede exportarse a formatos estándar. La interfaz web ofrece además botones de exportación a CSV y JSON (sección 8).

```
-- CSV
\copy (SELECT * FROM
autoaudit.history_by_column('public','clientes','saldo',NULL,NULL,NULL,FALSE)) TO 'saldo.csv'
CSV HEADER

-- JSON
\copy (SELECT jsonb_pretty(jsonb_agg(h)) FROM autoaudit.history_by_user('juan') h) TO
'juan.json'
```

7.3 Consulta directa sobre la bitácora

Para análisis a medida, la bitácora puede consultarse directamente (con el rol lector o administrador). Los índices sobre `operation_ts`, `id_user` y (`schema_name`, `table_name`, `column_name`) cubren estos patrones:

```
-- Todo lo que cambió una persona en una tabla, agrupado por operación
SELECT l.change_id, l.operation, l.operation_ts,
       jsonb_object_agg(l.column_name, jsonb_build_object('antes', l.old_value, 'despues',
l.new_value))
FROM autoaudit.audit_log l
JOIN autoaudit.audit_user u ON u.id_user = l.id_user
WHERE u.db_username = 'juan' AND l.table_name = 'clientes'
GROUP BY l.change_id, l.operation, l.operation_ts
ORDER BY l.operation_ts DESC;
```

8. Interfaz web

8.1 Arquitectura

La interfaz es una aplicación web sencilla en Python (Flask) que se conecta a PostgreSQL con psycopg 3. Su decisión de diseño más importante es que **inicia sesión con usuarios reales de PostgreSQL**: la aplicación abre la conexión con las credenciales del usuario, de modo que los permisos los hace cumplir la base de datos (roles `autoaudit_admin` y `autoaudit_reader`) y no la aplicación. Un usuario sin rol no puede ver nada, un lector solo consulta y un administrador configura.

Tabla 8. *Módulos de la interfaz.*

Pantalla	Ruta	Rol requerido	Función
Inicio de sesión	`/login`	Ninguno	Autentica contra PostgreSQL y determina el rol.

Pantalla	Ruta	Rol requerido	Función
Inicio	`/`	Cualquier sesión	Resumen de tablas auditadas, estado de triggers y tamaño estimado de la bitácora.
Configuración	`/config`	Administrador	Lista las tablas del diccionario de datos con su estado de auditoría.
Configurar tabla	`/config/<esquema>/<tabla>`	Administrador	Matriz de columnas × eventos; guarda, instala triggers o desactiva la tabla.
Triggers	`/triggers`	Administrador	Estado real de los triggers frente a `pg_trigger`.
Historial	`/history`	Lector o administrador	Consulta por usuario, columna o fila, con fechas y orden.
Exportación	`/history/export.csv`, `.json`	Lector o administrador	Descarga el resultado como archivo CSV o JSON.

8.2 Ejecución

Desde la carpeta del proyecto (primero se deben haber instalado los scripts y creado los usuarios de demostración con `sql/91_usuarios_demo.sql`):

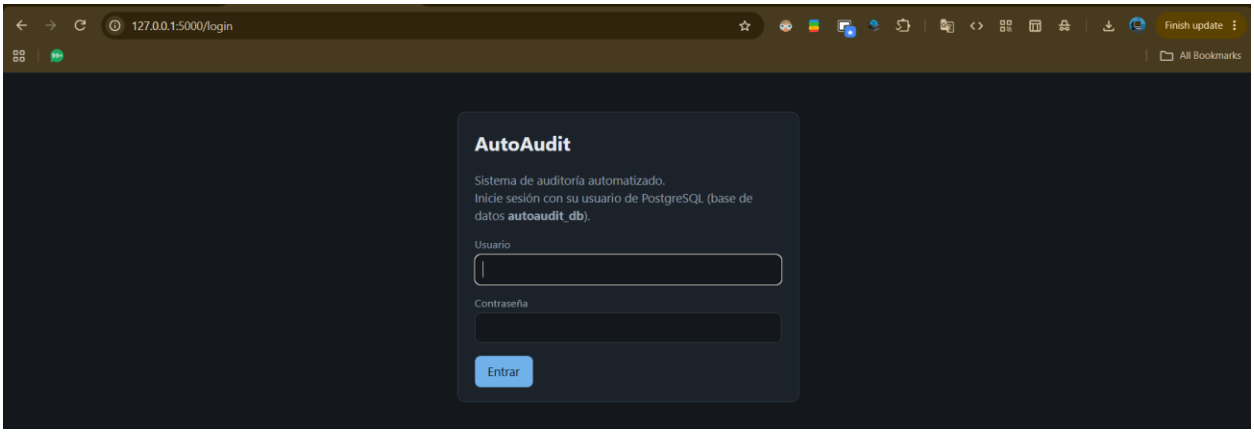
```
cd app
python -m venv venv
venv\Scripts\activate          :: en Linux: source venv/bin/activate
pip install -r requirements.txt
set AA_DB_NAME=autoaudit_db     :: en Linux: export AA_DB_NAME=autoaudit_db
python app.py
```

La aplicación queda disponible en `http://127.0.0.1:5000`. Variables opcionales: `AA_DB_HOST`, `AA_DB_PORT`, `AA_DB_NAME` y `AA_PORT`.

Tabla 9. *Usuarios de demostración (solo para pruebas).*

Usuario	Contraseña	Puede
`admin_demo`	`admin123`	Configurar la auditoría y consultar el historial.
`lector_demo`	`lector123`	Solo consultar y exportar el historial.
`usuario_demo`	`usuario123`	Nada de AutoAudit (usuario de negocio).

8.3 Pantallas



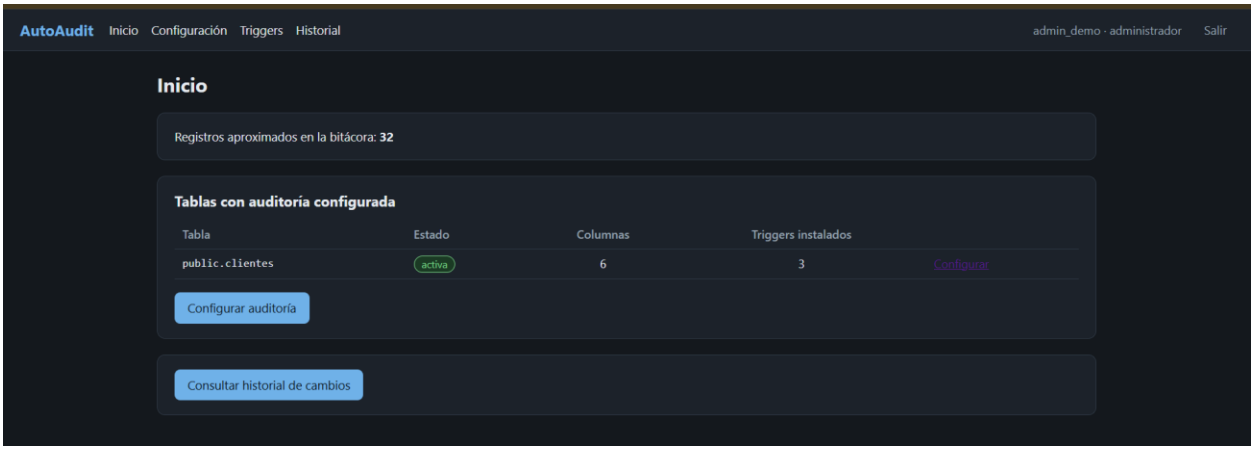


Figura 3. *Pantalla de inicio de sesión de AutoAudit*

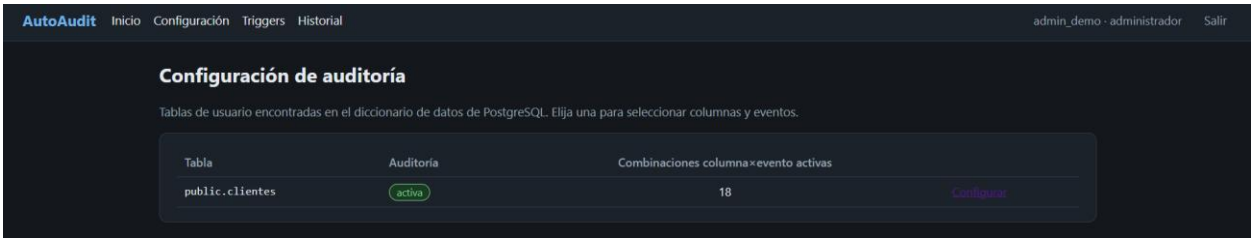


Figura 4. *Pantalla de Configuración: tablas de la base de datos y estado de auditoría*

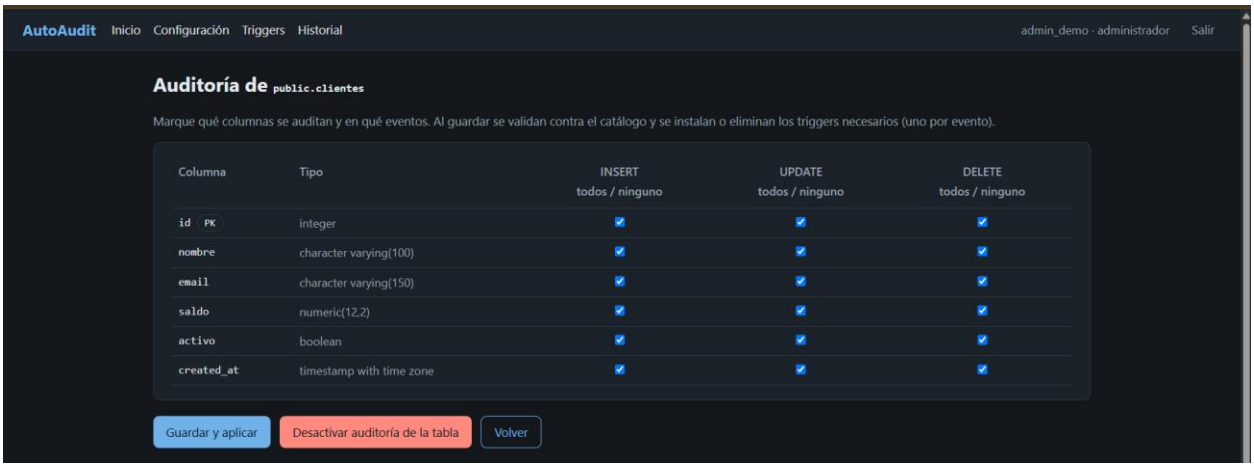


Figura 5. *Configuración de public.clientes: matriz de columnas × eventos y estado de triggers sincronizados*

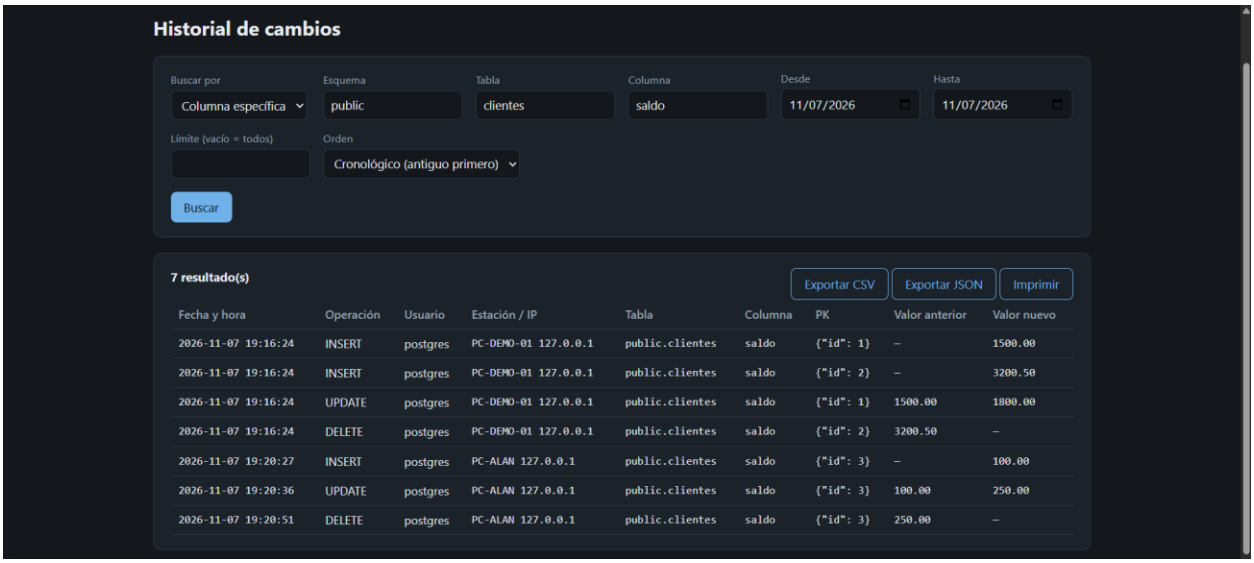


Figura 6. *Historial por columna (saldo) con valor anterior, valor nuevo, usuario, estación e IP*

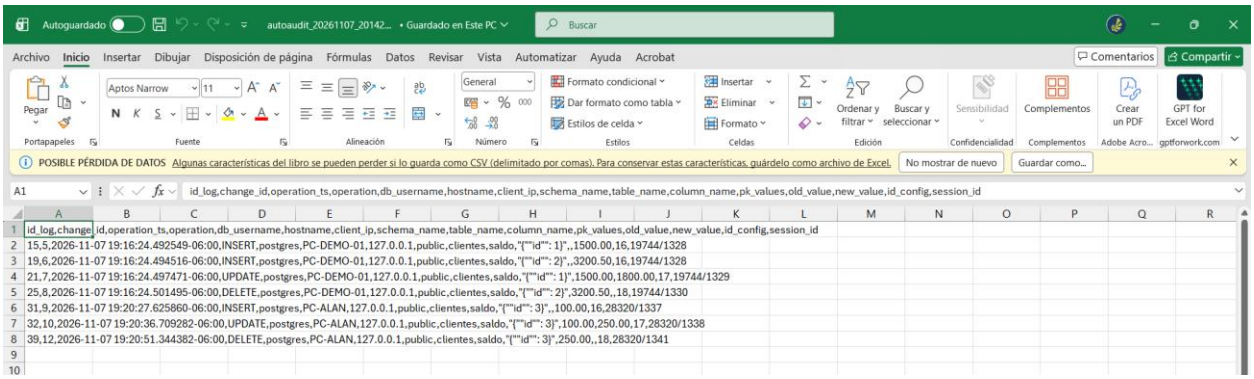


Figura 7. *Archivo CSV exportado abierto en una hoja de cálculo*

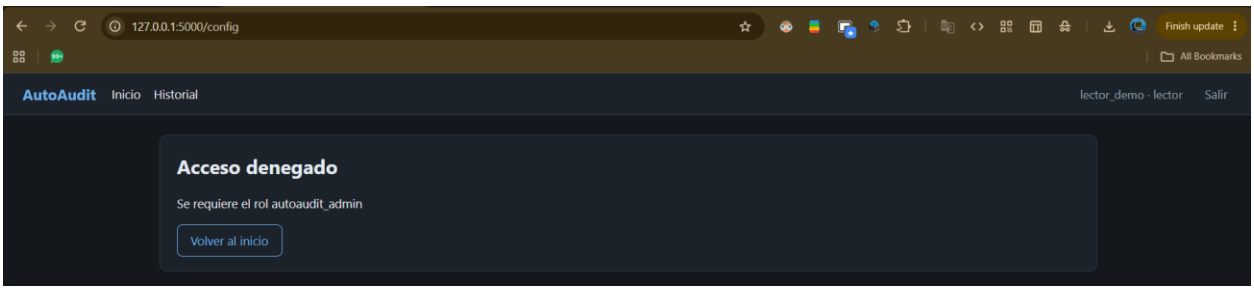


Figura 8. *Acceso denegado al usuario lector al intentar abrir /config*

8.4 Seguridad de la interfaz

- La contraseña de PostgreSQL nunca se guarda en el navegador: la cookie contiene solo un token aleatorio y las credenciales viven en la memoria del servidor.
- Todas las acciones que modifican datos son POST y exigen un token CSRF por sesión; sin él la petición se rechaza.

- Todas las consultas usan parámetros enlazados; las plantillas escapan el contenido automáticamente.
- Los errores de PostgreSQL (por ejemplo, privilegios insuficientes) se muestran como mensajes claros, sin exponer detalles internos.
- Para un despliegue real se recomienda servir la aplicación detrás de HTTPS y de un servidor WSGI; el servidor de desarrollo incluido es suficiente para fines académicos.

9. Seguridad y cumplimiento de restricciones

Tabla 10. *Restricciones del proyecto y cómo se cumplen.*

Restricción	Implementación
Solo usuarios administrativos configuran la auditoría	Cada función de configuración invoca `assert_admin()`. Además, `EXECUTE` está revocado a `PUBLIC` y otorgado solo a `autoaudit_admin`; los administradores no tienen permiso de escritura directa sobre las tablas, solo a través de las funciones.
La bitácora no admite escritura directa	Ningún rol tiene INSERT sobre `audit_log`; solo la función `SECURITY DEFINER` `fn_audit_generic` escribe en ella. Un trigger de protección bloquea UPDATE, DELETE y TRUNCATE incluso para el dueño.
Cada trigger maneja exclusivamente una tabla origen	`trigger_registry` es único por (tabla, evento); el generador crea el trigger sobre la tabla indicada y le asigna un nombre derivado de ella.
Validar existencia de tablas y columnas antes de activar	`register_table` y `register_column` consultan `pg_catalog` y rechazan lo que no existe; `install_triggers` vuelve a comprobarlo.
La bitácora preserva los datos aunque se elimine el registro fuente	No hay claves foráneas hacia tablas de negocio; `schema_name`, `table_name`, `column_name`, `pk_values`, `old_value` y `new_value` se guardan de forma independiente.
La IP respeta las políticas de privacidad	La bitácora completa es legible solo por los roles lector y administrador; `PUBLIC` no tiene acceso. La retención y el acceso a `client_ip` deben definirse según las políticas de privacidad aplicables a la organización.

Defensa en profundidad
Las funciones `SECURITY DEFINER` fijan su `search_path` (`pg_catalog, autoaudit, pg_temp`) para evitar la suplantación de objetos, el usuario se toma de `session_user` y la interfaz delega la autenticación en PostgreSQL.

10. Pruebas de validación

La validación combina pruebas automáticas en SQL (`tests/01_funcional.sql`), pruebas automáticas de la interfaz (`tests/03_interfaz.py`) y una validación manual guiada. Las pruebas SQL usan verificaciones `ASSERT` que detienen el script con error si algún resultado no coincide, y actúan con usuarios reales mediante `SET SESSION AUTHORIZATION`.

10.1 Pruebas funcionales en SQL

Tabla 11. *Casos de prueba de `01_funcional.sql`.*

Id	Caso de prueba	Resultado esperado	Resultado
T1	Un usuario sin rol administrador intenta configurar.	Error de privilegios insuficientes.	Aprobada
T2	Registrar tabla, columna y evento inexistentes.	Los tres intentos son rechazados con error claro.	Aprobada
T3	Activar auditoría de `nombre`, `email` y `saldo`.	3 triggers instalados; registro sincronizado con `pg_trigger`; PK detectada.	Aprobada
T4	**INSERT** de una fila.	1 fila de bitácora por columna auditada (3) con el mismo `change_id`; `old_value` NULL; usuario real, estación, `id_config` e `id_table`; `pk_values` solo con la PK; la columna no auditada no aparece.	Aprobada
T5	**UPDATE** de `saldo` y `email`; UPDATE de una columna no auditada; UPDATE sin cambio real.	Solo 2 filas de bitácora (las columnas que cambiaron) con valor antiguo y nuevo.	Aprobada
T6	**DELETE** de la fila.	La fila origen desaparece; la bitácora conserva los valores anteriores y `new_value` es NULL.	Aprobada
T7	Consultas por usuario, columna y fila, con orden y rango de fechas.	Conteos correctos y orden cronológico respetado.	Aprobada
T8	Intentar UPDATE, DELETE y TRUNCATE sobre la bitácora; leer o escribirla con un usuario normal.	Todos los intentos son rechazados.	Aprobada
T9	Desactivar el evento DELETE.	Se elimina su trigger, el registro queda sincronizado y los DELETE posteriores ya no se auditan.	Aprobada
T10	Instalar triggers sobre una tabla que ya no existe.	Error claro del generador.	Aprobada
T11	Eliminar la configuración de la tabla.	El historial se conserva; `id_config` e `id_table` pasan a NULL.	Aprobada

La ejecución en PostgreSQL 18.6 terminó con el mensaje `TODAS LAS PRUEBAS PASARON`.

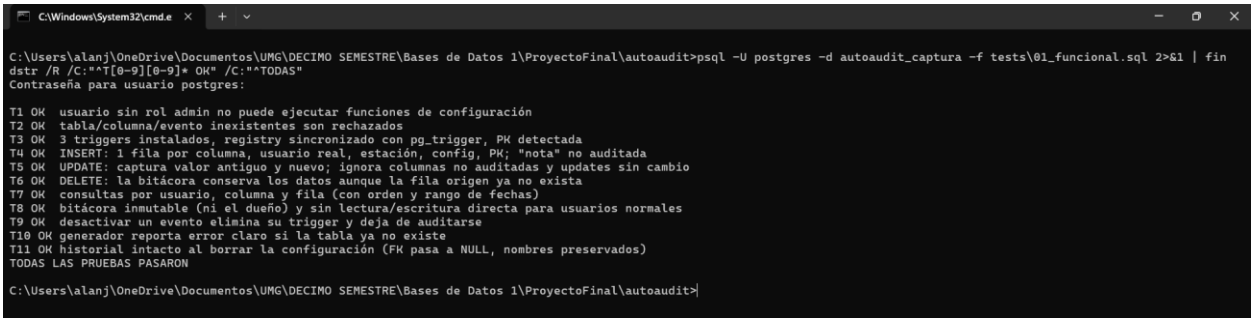


Figura 9. *Salida de tests/01_funcional.sql: T1 a T11 en OK y «TODAS LAS PRUEBAS PASARON»*

10.2 Pruebas automáticas de la interfaz

El script `tests/03_interfaz.py` ejercita la aplicación web contra una base real, con los usuarios de demostración, y realiza 25 verificaciones agrupadas así:

Grupo	Verificaciones
Inicio de sesión	Contraseña incorrecta rechazada; un usuario sin rol inicia sesión pero no accede a `/config`, `/history` ni `/triggers`.
Configuración (administrador)	Lista tablas del diccionario; rechaza POST sin token CSRF; guarda la configuración; se instalan 3 triggers; la pantalla refleja lo guardado; tabla inexistente devuelve 404; pantalla de triggers.
Historial (lector)	El lector no accede a configuración ni puede desactivar auditoría; consulta por usuario muestra valores antiguo y nuevo; consulta por columna en orden cronológico; consulta por fila; PK inválida muestra mensaje claro.
Exportación	CSV con 3 filas y primera operación INSERT; JSON con valores correctos; descarga como archivo adjunto.
Desactivación	Se eliminan los triggers; la bitácora se conserva.
Cierre de sesión	El logout invalida la sesión.

El script termina con `TODAS LAS PRUEBAS DE LA INTERFAZ PASARON`. Se ejecuta con `python tests\03_interfaz.py` desde la carpeta del proyecto, con el entorno virtual activo y la variable `AA_DB_NAME` apuntando a la base de pruebas.

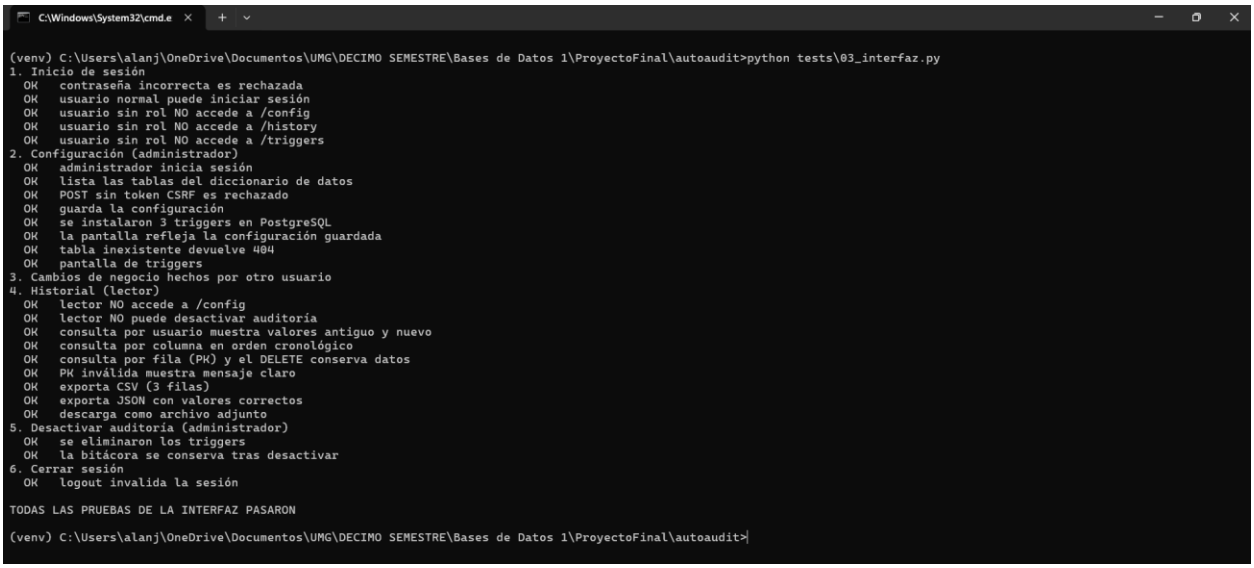


Figura 10. *Salida de tests/03_interfaz.py con todas las verificaciones en OK*

10.3 Validación manual guiada

Tabla 12. *Pasos de validación manual de la interfaz.*

Paso	Acción	Resultado esperado	Resultado
1	Como administrador, configurar `clientes` con `nombre`, `email` y `saldo` en los tres eventos.	«Configuración guardada», 3 triggers `INSTALLED` y «Sincronizado: sí».	Aprobada
2	Desde `psql` hacer INSERT, UPDATE de `saldo`, UPDATE de `activo` (no auditada) y DELETE.	Las cuatro sentencias se ejecutan sin error.	Aprobada

Paso	Acción	Resultado esperado	Resultado
3	Consultar el historial por columna `saldo`, por usuario y por fila.	INSERT, UPDATE y DELETE con valores antiguo y nuevo, usuario, hostname e IP; el cambio de `activo` no aparece.	Aprobada
4	Exportar a CSV y JSON.	Los archivos coinciden con lo mostrado en pantalla.	Aprobada
5	Entrar como lector y como usuario sin rol.	El lector no ve Configuración; `/config` muestra «Acceso denegado». El usuario sin rol no ve nada de AutoAudit.	Aprobada
6	Desactivar la auditoría de la tabla y hacer un nuevo UPDATE.	No se registra el cambio nuevo; el historial anterior se conserva.	Aprobada

10.4 Casos de prueba exigidos por el enunciado

Caso exigido	Dónde se cubre
INSERT con registro correcto en bitácora	T4 (sección 10.1)
UPDATE con captura de valor antiguo y nuevo	T5 (sección 10.1)
DELETE con preservación de datos en bitácora	T6 y T11 (sección 10.1)
Pruebas de rendimiento con volumen creciente de registros auditados	Sección 11

11. Pruebas de rendimiento

11.1 Objetivo y metodología

El criterio técnico del proyecto exige que los triggers no impacten de forma significativa el rendimiento de las operaciones normales. Para medirlo, el script `tests/02_rendimiento.sql` compara dos tablas idénticas: `perf_base` (sin auditoría) y `perf_aud` (con auditoría de `nombre`, `email` y `saldo` en INSERT, UPDATE y DELETE). Para cada volumen (1 000, 10 000, 100 000 y 1 000 000 de filas) se ejecutan cuatro operaciones masivas, cada una como una sola sentencia con su `COMMIT` incluido en la medición:

- **INSERT** de N filas con `generate_series`.
- **UPDATE** de una columna auditada (`saldo`) en las N filas.
- **UPDATE** de una columna **no** auditada (`nota`) en las N filas.
- **DELETE** de las N filas.

El script registra el tiempo en milisegundos y cuántas filas se escribieron en la bitácora, y al final mide las consultas históricas sobre la bitácora acumulada.

11.2 Entorno de prueba

Elemento	Valor
Gestor de base de datos	PostgreSQL 18.6
Sistema operativo	Windows
Equipo	Intel Core i5 10210U CPU, 16GB de Ram, SSD: KBG40 512GB

Elemento	Valor
Configuración de PostgreSQL	Valores por defecto de la instalación

11.3 Resultados

Tabla 13. *Tiempo por operación sin y con auditoría, resultado de la ejecución de* `02_rendimiento.sql`.

Filas	Operación	Sin auditoría (ms)	Con auditoría (ms)	Veces más lento	µs extra por fila	Filas en bitácora
1 000	INSERT	11.1	657.3	59.2	646.2	3 000
1 000	UPDATE col. auditada (saldo)	12.0	366.9	30.6	354.9	1 000
1 000	UPDATE col. NO auditada (nota)	9.6	71.2	7.4	61.6	0
1 000	DELETE	0.9	530.0	588.9	529.1	3 000
10 000	INSERT	69.0	7 101.7	102.9	703.3	30 000
10 000	UPDATE col. auditada (saldo)	102.1	5 287.7	51.8	518.6	10 000
10 000	UPDATE col. NO auditada (nota)	154.8	1 431.3	9.2	127.7	0
10 000	DELETE	16.3	8 402.0	515.5	838.6	30 000
100 000	INSERT	745.1	85 467.8	114.7	847.2	300 000
100 000	UPDATE col. auditada (saldo)	1 412.3	54 858.7	38.8	534.5	100 000
100 000	UPDATE col. NO auditada (nota)	1 612.9	11 856.8	7.4	102.4	0
100 000	DELETE	174.0	81 450.0	468.1	812.8	300 000
1 000 000	INSERT	8 578.0	885 428.8	103.2	876.9	3 000 000
1 000 000	UPDATE col. auditada (saldo)	18 875.2	483 558.3	25.6	464.7	1 000 000
1 000 000	UPDATE col. NO auditada (nota)	10 863.3	66 643.9	6.1	55.8	0
1 000 000	DELETE	2 759.8	432 535.5	156.7	429.8	3 000 000

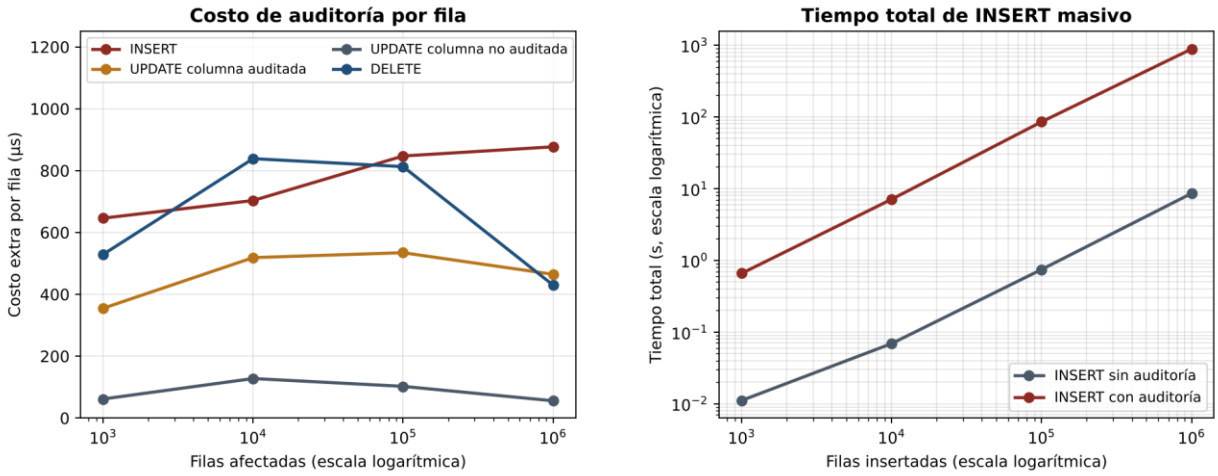


Figura 11. *Costo de auditoría por fila (izquierda) y tiempo total de INSERT masivo con y sin auditoría (derecha), ambos en escala logarítmica.*

11.4 Tamaño de la bitácora

Al finalizar la prueba la bitácora acumuló las filas generadas por todos los volúmenes (3 333 000 de INSERT, 1 111 000 de UPDATE de `saldo` y 3 333 000 de DELETE):

Medida	Valor
Filas en `audit_log`	7 777 000
Tamaño total con índices	3 205 MB
Tamaño de la tabla	1 530 MB
Tamaño de los índices (por diferencia)	aprox. 1 675 MB
Tamaño medio por fila de bitácora	aprox. 430 bytes (incluyendo índices)

11.5 Consultas históricas con 7.7 millones de filas

Consulta	Tiempo	Observación
Por usuario (últimas 1 000 filas)	215.5 ms	El usuario de la prueba realizó todos los cambios; se debe ordenar un volumen muy grande.
Por usuario, última hora, orden cronológico	175.9 ms	Mismo caso con filtro de fechas.
Por columna (`perf_aud.saldo`, últimas 1 000 filas)	3.1 ms	Cubierta por el índice (esquema, tabla, columna, fecha).
Por fila (PK `{"id": 500}`)	1 069.3 ms	No existe índice sobre `pk_values` (compromiso deliberado, ver 11.6).

11.6 Análisis

- **Escala casi lineal.** El costo extra por fila en INSERT pasa de unos 650 µs con 1 000 filas a unos 880 µs con un millón. Sube moderadamente porque los índices de la bitácora crecen, pero no se dispara.
- **Operaciones normales.** Una operación sobre una sola fila cuesta menos de 1 ms adicional, lo que es imperceptible en el uso transaccional habitual. Cada INSERT genera tres filas de bitácora (una por columna auditada), es decir, unos 290 µs por fila de bitácora.
- **Cargas masivas.** En porcentaje el costo parece grande (de 25 a 590 veces) porque sin auditoría estas operaciones son casi instantáneas. En términos absolutos, un millón de INSERT pasa de 8.6 s (unas 116 000 filas/s) a 885 s (unas 1 130 filas/s). Es el precio de registrar siete millones de cambios con varios índices.
- **UPDATE de columnas no auditadas.** Cuesta entre 56 y 128 µs por fila: el trigger se dispara y descubre que no hay nada que registrar. Podría eliminarse con triggers `AFTER UPDATE OF <columnas>`, pero se descartó porque exigiría reinstalar los triggers ante cada cambio de configuración y, si se olvida, una columna quedaría sin auditar sin aviso. Se prefirió la corrección sobre la velocidad.
- **Consulta por fila (PK).** Un índice GIN sobre `pk_values` la volvería casi instantánea, pero encarecería cada escritura. Queda como índice opcional comentado en el script.
- **Sin índices GIN sobre los valores.** El diseño usa índices B-tree y no GIN sobre `old_value` y `new_value`, lo que reduce el costo de cada escritura.

- **Dependencia del equipo.** Los tiempos absolutos dependen del hardware (en especial del disco); lo relevante es el comportamiento lineal y la proporción.

11.7 Recomendaciones de operación

- Auditar solo las columnas necesarias: el costo crece con la cantidad de columnas configuradas.
- Para cargas masivas planificadas, desactivar la auditoría de la tabla (`disable_audit`) durante la carga y reactivarla al terminar, dejando constancia de la ventana sin auditoría.
- Almacenar la bitácora en un disco rápido y planificar su retención y archivado, dado que crece unos 430 bytes por cambio auditado.
- Evaluar el particionamiento de `audit_log` por fecha si el volumen lo justifica (ver sección 13).

Conclusión sobre el criterio de aceptación
Para operaciones normales (una o pocas filas por transacción) el impacto es inferior a 1 ms por operación y se considera no significativo. Para cargas masivas el impacto es considerable y se documenta junto con su mitigación (sección 11.7).

12. Verificación de los criterios de aceptación

Tabla 14. *Criterios de aceptación del enunciado y su evidencia.*

Criterio	Tipo	Cómo se cumple	Evidencia
El sistema genera triggers válidos que se ejecutan correctamente para cada operación configurada	Funcional	Generador `install_triggers` a partir de `audit_config`.	T3 a T6; §6
Cada cambio queda registrado con todos los campos requeridos	Funcional	`audit_log` con valores antiguo y nuevo, usuario, estación, IP, fecha y hora, PK y configuración.	T4, T5; §3.8
Consultas por usuario retornan el historial completo	Funcional	`history_by_user` con índices por usuario y fecha.	T7; §7
Consultas por columna muestran todos los cambios de esa columna	Funcional	`history_by_column`; índice por (esquema, tabla, columna, fecha).	T7; §7, 11.5
Los triggers no impactan el rendimiento de forma significativa en operaciones normales	Técnico	Menos de 1 ms adicional por operación; costo lineal; impacto documentado en cargas masivas.	§11
La tabla de configuración refleja el estado actual	Técnico	`is_enabled`, `is_active`, `is_installed` y la vista `v_trigger_status`.	T3, T9; §5.6
La bitácora mantiene integridad referencial con las tablas origen configuradas	Técnico	FK `id_table` → `audited_table` (SET NULL) y `id_config` → `audit_config`; los nombres se preservan.	T11; §2.3
El sistema maneja adecuadamente los errores en la generación o ejecución de triggers	Técnico	Captura por evento con `last_error`; validaciones previas; política de integridad en la ejecución.	T2, T10; §6.9

13. Limitaciones y trabajo futuro

13.1 Limitaciones conocidas

- No se audita `TRUNCATE` (el enunciado se limita a INSERT, UPDATE y DELETE).

- Un superusuario de PostgreSQL puede desactivar triggers; AutoAudit protege la bitácora, pero no puede impedir la acción de un superusuario sobre los triggers de la tabla de negocio.
- Si la conexión es por socket local de Unix, `client_ip` queda NULL.
- El nombre de host y el sistema operativo de la estación los debe informar la aplicación cliente; el servidor solo conoce la IP.
- La consulta por fila (PK) no tiene índice por defecto.
- En cargas masivas el costo de la auditoría es elevado (sección 11).
- Las sesiones de la interfaz viven en la memoria de un solo proceso y no hay HTTPS: se trata de una aplicación de uso académico.
- No hay un proceso automático de retención o archivado de la bitácora.

13.2 Trabajo futuro

- Triggers `AFTER UPDATE OF <columnas>` con reinstalación automática ante cambios de configuración, para eliminar el costo de los UPDATE sobre columnas no auditadas.
- Particionamiento de `audit_log` por fecha, con política de retención y archivado.
- Triggers a nivel de sentencia con tablas de transición para reducir el costo en cargas masivas.
- Enmascaramiento opcional de `client_ip` y definición formal de la política de acceso a datos personales.
- Auditoría de `TRUNCATE` y de cambios de estructura mediante triggers de eventos.
- Despliegue de la interfaz con HTTPS, servidor WSGI y almacenamiento de sesiones persistente.

14. Conclusiones

AutoAudit cumple el objetivo del proyecto: permite configurar qué tablas, columnas y eventos auditar a partir del diccionario de datos, genera e instala automáticamente los triggers correspondientes y registra cada cambio en una bitácora centralizada con valores antiguos y nuevos, usuario, estación, IP y marca temporal, consultable por usuario, columna o fila.

El diseño prioriza la integridad de la auditoría: la bitácora es de solo-append, la configuración solo puede modificarla un administrador mediante funciones que validan contra el catálogo, el usuario se registra de forma correcta incluso dentro de funciones `SECURITY DEFINER`, y no se permite ningún cambio de negocio sin su registro. La separación entre configuración, triggers instalados y bitácora, junto con la vista de estado real de los triggers, mantiene la configuración consistente con lo que existe en la base de datos.

Las pruebas automáticas y manuales confirman el comportamiento esperado en INSERT, UPDATE y DELETE, la preservación del historial y las restricciones de seguridad. Las pruebas de rendimiento muestran un costo lineal y moderado por operación individual, y un costo considerable en cargas masivas, que se documenta junto con sus mitigaciones. Con ello, el sistema ofrece una base sólida y extensible para auditar bases de datos PostgreSQL.

Anexo A. Función genérica fn_audit_generic

Código completo de la función que ejecutan todos los triggers de auditoría, tal como aparece en `sql/05_trigger_function.sql`.

```
CREATE OR REPLACE FUNCTION autoaudit.fn_audit_generic()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog, autoaudit, pg_temp
AS $$
DECLARE
    v_id_table BIGINT;
    v_pk_cols  TEXT[];
    v_old      JSONB;
    v_new      JSONB;
    v_pk       JSONB;
    v_cfg_ids  BIGINT[];
    v_cols     TEXT[];
    v_user     BIGINT;
    v_station  BIGINT;
    v_change   BIGINT;
    v_session  TEXT;
BEGIN
    SELECT t.id_table, t.pk_columns
    INTO v_id_table, v_pk_cols
    FROM autoaudit.audited_table t
    WHERE t.schema_name = TG_TABLE_SCHEMA
      AND t.table_name  = TG_TABLE_NAME
      AND t.is_active;

    IF v_id_table IS NULL THEN
        RETURN NULL;                       -- tabla no activa: nada que auditar
    END IF;

    IF TG_OP IN ('UPDATE', 'DELETE') THEN v_old := to_jsonb(OLD); END IF;
    IF TG_OP IN ('INSERT', 'UPDATE') THEN v_new := to_jsonb(NEW); END IF;

    -- Configuraciones activas para este evento (y, en UPDATE, solo columnas que cambiaron)
    SELECT array_agg(cfg.id_config  ORDER BY ac.column_name),
           array_agg(ac.column_name ORDER BY ac.column_name)
    INTO v_cfg_ids, v_cols
    FROM autoaudit.audit_config  cfg
    JOIN autoaudit.audited_column ac ON ac.id_column = cfg.id_column
    JOIN autoaudit.audit_event    ev ON ev.id_event  = cfg.id_event
    WHERE ac.id_table  = v_id_table
      AND ac.is_active
      AND cfg.is_enabled
      AND ev.event_code = TG_OP
      AND (TG_OP <> 'UPDATE'
           OR (v_old -> ac.column_name) IS DISTINCT FROM (v_new -> ac.column_name));

    IF v_cols IS NULL THEN
        RETURN NULL;                       -- nada auditable cambió
    END IF;

    -- PK de la fila (valores anteriores en UPDATE/DELETE, nuevos en INSERT)
    IF cardinality(v_pk_cols) > 0 THEN
        SELECT jsonb_object_agg(k, COALESCE(COALESCE(v_old, v_new) -> k, 'null'::jsonb))
        INTO v_pk
        FROM unnest(v_pk_cols) AS k;
    END IF;

    v_user    := autoaudit.resolve_user();
    v_station := autoaudit.resolve_station();
    v_change  := nextval('autoaudit.seq_change_id');
    v_session := pg_backend_pid()::text || '/' || pg_current_xact_id()::text;

    INSERT INTO autoaudit.audit_log (
        change_id, id_config, id_table, id_user, id_station, operation, operation_ts,
        schema_name, table_name, column_name, pk_values, old_value, new_value,
```

```
        client_ip, session_id)
    SELECT v_change, u.id_config, v_id_table, v_user, v_station, TG_OP, clock_timestamp(),
           TG_TABLE_SCHEMA, TG_TABLE_NAME, u.col, v_pk,
           v_old -> u.col,                 -- NULL en INSERT
           v_new -> u.col,                 -- NULL en DELETE
           inet_client_addr(), v_session
    FROM unnest(v_cfg_ids, v_cols) AS u(id_config, col);

    RETURN NULL;                           -- valor ignorado en triggers AFTER
END;
$$;
```

Anexo B. Repositorio y estructura del proyecto

Repositorio de código fuente: https://github.com/alansb11/autoaudit

Tabla 15. *Estructura del repositorio.*

Carpeta / archivo	Contenido
`sql/`	Scripts de instalación (`01` a `08`, `install.sql`), consultas al diccionario de datos, generador de triggers, consultas históricas y scripts opcionales de demostración (`90`, `91`).
`tests/01_funcional.sql`	Pruebas funcionales de INSERT, UPDATE, DELETE, permisos y errores.
`tests/02_rendimiento.sql`	Pruebas de rendimiento con volumen creciente.
`tests/03_interfaz.py`	Pruebas automáticas de la interfaz web.
`app/`	Interfaz web: `app.py`, `requirements.txt` y plantillas HTML.
`README.md`	Guía rápida de instalación, ejecución y decisiones de diseño.

B.1 Resumen de comandos

```
:: 1. Crear la base e instalar AutoAudit
psql -U postgres -c "CREATE DATABASE autoaudit_db;"
psql -U postgres -d autoaudit_db -f sql\install.sql

:: 2. (Opcional) Demostración y usuarios de ejemplo
psql -h 127.0.0.1 -U postgres -d autoaudit_db -f sql\90_demo.sql
psql -U postgres -d autoaudit_db -f sql\91_usuarios_demo.sql

:: 3. Pruebas funcionales (en una base aparte: dejan registros en la bitácora)
psql -U postgres -c "CREATE DATABASE autoaudit_test;"
psql -U postgres -d autoaudit_test -f sql\install.sql
psql -U postgres -d autoaudit_test -f tests\01_funcional.sql

:: 4. Pruebas de rendimiento (en una base aparte; eliminarla al terminar)
psql -U postgres -c "CREATE DATABASE autoaudit_perf;"
psql -U postgres -d autoaudit_perf -f sql\install.sql
psql -U postgres -d autoaudit_perf -f tests\02_rendimiento.sql > resultados_rendimiento.txt
psql -U postgres -c "DROP DATABASE autoaudit_perf;"

:: 5. Interfaz web
cd app
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
set AA_DB_NAME=autoaudit_db
python app.py
```